

LAPORAN PRAKTIKUM

STRUKTUR DATA

MODUL KE 6

Sorting



Disusun Oleh:

Nama : Noufal Aji Prasetyo

NPM : 2340506059

Kelas : Rombel 3

Program Studi S1 Teknologi Informasi

Fakultas Teknik, Universitas Tidar

Genap 2024/2025

A. Tujuan Praktikum

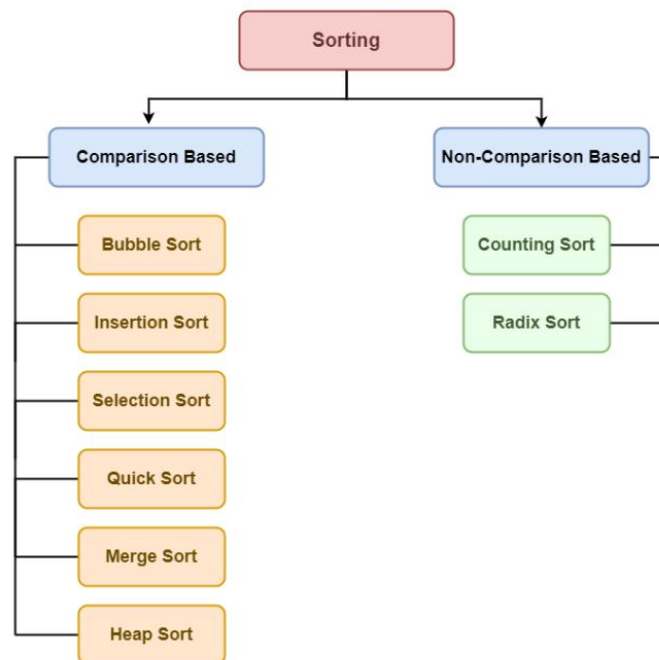
Agar setiap mahasiswa tahu cara mengaplikasikan struktur data sorting dalam python. Dan mahasiswa mampu memahami algoritma struktur data sorting, macam-macam penerapan, dan fungsi pada setiap code

B. Dasar Teori

Pengurutan mengacu pada penataan ulang larik atau daftar elemen tertentu menurut operator perbandingan pada elemen tersebut. Operator perbandingan digunakan untuk menentukan elemen baru dalam struktur data masing-masing.

Jenis Teknik Pengurutan

1. Berbasis Perbandingan : membandingkan elemen dalam algoritma pengurutan berbasis perbandingan
2. Berbasis non-perbandingan : tidak membandingkan elemen dalam algoritma pengurutan berbasis non-perbandingan



Algoritma pengurutan penting dalam Ilmu Komputer karena mengurangi kompleksitas suatu masalah. Ada berbagai macam penerapan algoritma ini, termasuk algoritma pencarian, algoritma database, metode pembagian dan penaklukkan, dan algoritma struktur data.

Beberapa Algoritma pengurutan yang paling umum adalah :

1. Selection Sort

Selection sort adalah Teknik pengurutan lain dimana kita menemukan elemen minimum di setiap iterasi dan menempatkannya dalam array yang dimulai indeks pertama. Dengan demikian, pengurutan pilihan juga dibagi menjadi subarray yang terurut dan tidak terurut

2. Bubble Sort

Bubble sort adalah algoritma pengurutan paling sederhana yang bekerja dengan menukar elemen yang berdekatan berulang kali jika urutannya salah. Algoritma ini tidak cocok untuk kumpulan data besar karena kompleksitas waktunya rata-rata dan kasus terburuknya cukup tinggi.

3. Insertion sort

Insertion sort adalah algoritma pengurutan sederhana yang cara kerjanya mirip dengan cara mengurutkan kartu remi di tangan. Array sebenarnya dibagi menjadi bagian yang diurutkan dan tidak diurutkan. Nilai dari bagian yang tidak diurutkan diambil dan ditempatkan pada posisi yang benar di bagian yang diurutkan

4. Merge sort

Merge sort merupakan algoritma pengurutan yang didasarkan pada paradigma Divide and Conquer. Dalam algoritma ini, array awalnya dibagi menjadi dua bagian yang sama dan kemudian digabungkan secara terurut. Algoritma merge sort merupakan implementasi dari Teknik membagi dan menaklukkan. Dengan demikian, ini diselesaikan dalam 3 langkah

- a. Divide : Pada Langkah ini, array membagi dirinya secara rekursif menjadi sub-array hingga kasus dasar tercapai
- b. Conquer : disini, sub-array diurutkan menggunakan rekursi
- c. Combine : Langkah ini menggunakan fungsi merge() untuk menggabungkan sub-array ke dalam array terurut

5. Quick Sort

Quicksort merupakan algoritma pengurutan berdasarkan pendekatan Divide and Conquer dimana sebuah array dibagi menjadi beberapa subarray dengan memilih elemen pivot (elemen yang dipilih dari array)

- a. Saat membagi array, elemen pivot harus diposisikan sedemikian rupa sehingga elemen yang lebih kecil dari pivot berada di sisi kiri, dan elemen yang lebih besar dari pivot berada di sisi kanan pivot
- b. Subarray kiri dan kanan juga dibagi menggunakan pendekatan yang sama. Proses ini berlanjut hingga setiap subarray berisi satu elemen.
- c. Pada titik ini, elemen sudah diurutkan. Terakhir, elemen digabungkan untuk membentuk array yang diurutkan.

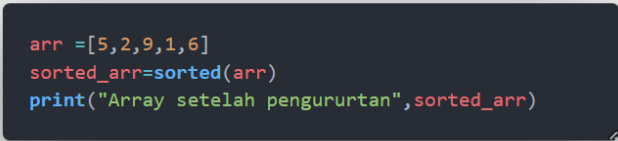
6. Heap Sort

Heapsort adalah Teknik pengurutan berbasis perbandingan berdasarkan struktur data Binary Heap. Ini mirip dengan pengurutan seleksi dimana kita pertama kali menemukan elemen minimum dan menempatkan elemen minimum di awal. Ulangi proses yang sama dengan elemen lainnya

7. Counting Sort

Counting sort merupakan Teknik pengurutan berdasarkan kunci antara rentang tertentu. Ini akan bekerja menghitung jumlah objek yang memiliki nilai kunci berbeda (semacam hashing). Kemudian lakukan beberapa aritmatika untuk menghitung posisi setiap objek pada urutan keluaran.

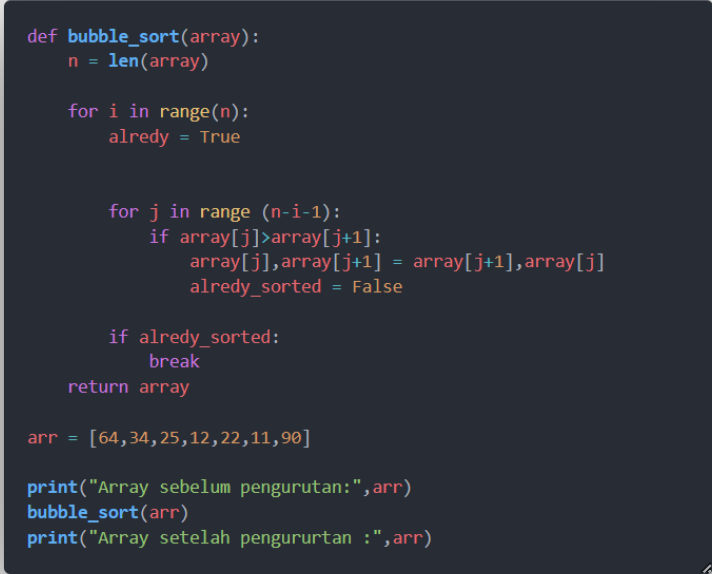
C. Hasil dan Pembahasan



```
arr =[5,2,9,1,6]
sorted_arr=sorted(arr)
print("Array setelah pengurutan",sorted_arr)
```

Gambar1.1

- Mendefinisikan array dengan nilai yang akan diurutkan
- Menggunakan fungsi sorted untuk mengurutkan array. Fungsi sorted mengembalikan array yang sudah diurutkan dari array yang diberikan
- Mencetak array yang sudah diurutkan ke layer yang berisi array yang sudah diurutkan



```
def bubble_sort(array):
    n = len(array)

    for i in range(n):
        already = True

        for j in range (n-i-1):
            if array[j]>array[j+1]:
                array[j],array[j+1] = array[j+1],array[j]
                already_sorted = False

        if already_sorted:
            break
    return array

arr = [64,34,25,12,22,11,90]

print("Array sebelum pengurutan:",arr)
bubble_sort(arr)
print("Array setelah pengurutan :",arr)
```

Gambar1.2

- Membuat fungsi bubble sort. Fungsi ini menerima 1 parameter yaitu array yang akan diurutkan
- Variable n ini menyimpan panjang array yang akan diurutkan
- Untuk looping pertama dilakukan untuk mengetahui proses melalui setiap elemen di dalam array. Ini dilakukan menggunakan loop for dengan variable i
- Variable already_sorted ini digunakan untuk melacak apakah array sudah terurut atau belum. Pada gambar diatas diasumsikan bahwa array sudah terurut karena already_sorted = true

- Untuk looping yang kedua. Ada proses yang dilakukan menggunakan loop for dengan variable j. ini digunakan untuk membandingkan elemen berpasangan dan menukar posisinya.
- Selajutnya dilakukan pengecekan apakah perlu dilakukan pertukaran. Setiap pasangan elemen berurutan dibandingkan, jika elemen pertama lebih besar dari elemen kedua maka ditukar posisinya. Selain itu variable `already_sorted` diganti menjadi false ini digunakan untuk menandakan bahwa array belum terurut
- Dihentikan prosesnya jika array sudah terurut. Jika selama proses tidak ada pertukaran yang dilakukan maka pengulangan dihentikan menggunakan `break`
- Setelah semua proses selesai, array yang telah diurutkan dikembalikan



```
def insertion_sort(array):
    for i in range(1, len(array)):
        key_item = array[i]

        j=i-1

        while j >= 0 and array[j]> key_item:
            array[j+1] = array[j]
            j-=1

        array[j+1]= key_item
    return array

arr = [64,34,25,12,22,11,90]

print("Array sebelum pengurutan:",arr)
bubble_sort(arr)
print("Array setelah pengurutan :",arr)
```

Gambar 1.3

- Insertion_sort ini adalah fungsi utama untuk melakukan pengurutan menggunakan algoritma insertion sort. Fungsi ini menerima satu parameter yaitu array yang akan diurutkan
- Looping untuk setiap elemen kecuali elemen pertama. Proses dimulai dari indeks ke 1
- Pada setiap proses, elemen yang sedang diproses disimpan dalam variable `key_item`
- Variable j diatur sebagai indeks untuk memindahkan elemen yang lebih besar dari `key_item` ke posisi yang sesuai
- Didalam looping while setiap elemen yang lebih besar dari `key_item` dipindahkan satu persatu keposisi kanan
- Setelah semua elemen yang lebih besar dari `key_item` dipindahkan ke kanan, maka `key_item` disisipkan pada posisi yang sesuai
- Setelah semua proses selesai, array yang telah diurutkan dikembalikan

```

def merge(left,right):
    if len(left) == 0:
        return right
    if len(right) == 0:
        return left
    result = []
    index_left = index_right = 0
    while len(result) < len(left) + len(right):
        if left[index_left] <= right[index_right]:
            result.append(left[index_left])
            index_left += 1
        else:
            result.append(right[index_right])
            index_right += 1
        if index_right == len(right):
            result += left[index_left:]
            break
        if index_left == len(left):
            result += right[index_right:]
            break
    return(result)

def merge_sort(array):
    if len(array) < 2:
        return(array)
    midpoint = len(array)//2
    return merge(
        left=merge_sort(array[:midpoint]),
        right=merge_sort(array[midpoint:])
    )
arr = [64,34,25,12,22,11,90]

print("Array sebelum pengurutan:",arr)
bubble_sort(arr)
print("Array setelah pengurutan :",arr)

```

Gambar 1.4

- Membuat fungsi merge(left,right) ini adalah fungsi untuk menggabungkan dua bagian array yang sudah terurut menjadi satu array yang terurut. Fungsi ini menerima dua parameter left dan right
- Dua blok if pertama memeriksa jika salah satu dari dua array kosong, maka fungsi langsung mengembalikan bagian array yang tidak kosong
- Variable result akan menyimpan array hasil penggabungan dari dua bagian array yang sudah terurut
- Variable index_left dan index_right untuk menyimpan indeks dari elemen yang sedang diproses di masing masing array
- Looping while untuk menggabungkan dua bagian array. Elemen dari dua array dibandingkan satu persatu dan elemen yang lebih kecil dimasukkan ke dalam array
- Setelah bagian array sudah diproses, sisa elemen dari bagian array yang belum diproses dimasukkan ke dalam array
- Merge_sort(array) ini adalah fungsi untuk melakukan pengurutan menggunakan merger sort
- Jika Panjang array kurang dari 2 maka array sudah terurut, maka fungsi langsung mengembalikan array tersebut
- Variable midpoint digunakan untuk membagi array menjadi dua bagian
- Fungsi merge_sort dipanggil untuk mengurutkan bagian array
- Hasil pengurutan dari kedua array yang sudah terurut diproses menggunakan fungsi merge untuk menggabungkan menjadi satu array

D. Latihan

```
def quicksort(arr):  
    if len(arr) <= 1:  
        return arr  
    else:  
        pivot = arr[-1]  
        left = [x for x in arr[:-1] if x < pivot]  
        right = [x for x in arr[:-1] if x >= pivot]  
        return quicksort(left) + [pivot] + quicksort(right)  
    )  
  
# Example usage  
arr = [64,34,25,12,22,11,90]  
sorted_arr = quicksort(arr)  
print("Pengurutan Array dari yang terkecil:")  
print(sorted_arr)
```

Gambar 1.5

- Pertama kita membuat fungsi utama untuk melakukan pengurutan menggunakan algoritma quicksort. Fungsi ini menerima satu parameter yaitu array yang akan diurutkan
- Yang kedua kita memeriksa Panjang array. Jika panjangnya kurang dari atau sama dengan 1, maka array tersebut sudah terurut dan langsung dikembalikan
- Ketiga kita menentukan pivot. Jika Panjang array lebih dari 1, maka elemen terakhir dari array -1 dipilih sebagai pivot
- Ke empat membagi array menjadi dua, left berisi elemen yang lebih kecil dari pivot dan right berisi elemen yang lebih besar dari pivot atau sama dengan pivot.
- Ke lima pengurutan secara rekursif. Setelah dibagi array dilakukan fungsi quicksort yang dipanggil secara rekursif untuk mengurutkan kedua bagian array left dan right
- Ke enam menggabungkan hasil dari pengurutan kedua bagian array left dan right dengan pivot
- Dan hasilnya dapat dilihat pada gambar dibawah

```
Pengurutan Array dari yang terkecil:  
[11, 12, 22, 25, 34, 64, 90]
```

Gambar 1.6

E. Kesimpulan

Kesimpulan dari praktikum di atas adalah mengetahui konsep konsep dasar terkait struktur data pengurutan dan bagaimana cara menggunakan nya

F. Referensi

- <https://www.geeksforgeeks.org/sorting-algorithms/>